Research Digest on Engineering Management and Social Innovations
Available on: http://rdems.in DOI: 10.46647/rdems0205067 Volume: 02 Issue: 05, May, 2026
p-ISSN: 3117-4418 e-ISSN: 3117-4426

AI Powered code Reviewer and Detector

[1]A.E. Raju, [2]Gajula Viswanath
[1]Assistant Professor, Department of CSE, Dr. K.V. Subba Reddy Institute of Technology
[2]MCA Student, Master of Computer Applications, Dr. K. V. Subba Reddy Institute of Technology

ABSTRACT
The project titled **"AI Powered Code Reviewer and Detector"** presents an intelligent system designed to automatically analyze and evaluate programming code using machine learning techniques. In modern software development, identifying bugs and security vulnerabilities at an early stage is crucial to improving code quality and reducing development time. This system leverages Natural Language Processing (NLP) with TF-IDF vectorization to convert source code into numerical features and applies multiple machine learning algorithms such as Logistic Regression, Support Vector Machine, Random Forest, Decision Tree, Naive Bayes, and K-Nearest Neighbors to classify code as either clean or buggy. The model with the highest accuracy is selected and stored for future predictions, ensuring optimal performance. Additionally, the system incorporates a security analysis module that detects risky patterns such as the use of functions like eval() and exec(), which may lead to potential vulnerabilities. The application is developed using the Flask web framework, providing a user-friendly interface that includes user registration, login authentication, code submission, prediction results, and history tracking of analyzed code. Furthermore, a comparative accuracy visualization of all trained models is generated to help understand model performance. This system enhances code review efficiency by automating bug detection and security checking, thereby assisting developers in writing safer and more reliable code while reducing manual effort and time consumption.

Keywords: Artificial Intelligence, Code Review, Bug Detection, Static Code Analysis, Machine Learning, Deep Learning, Software Quality Assurance, Automated Code Inspection, Vulnerability Detection, Source Code Analysis, Error Detection, Secure Coding, Code Optimization, Software Testing, Intelligent Debugging

I. INTRODUCTION

The project **"AI Powered Code Reviewer and Detector"** is developed to assist programmers in automatically analyzing source code for bugs and potential security vulnerabilities using machine learning techniques. In traditional software development, code review is a manual, time-consuming, and error-prone process that requires experienced developers to carefully inspect code for logical errors, inefficiencies, and unsafe practices. With the increasing complexity of software systems, there is a growing need for intelligent tools that can automate this process and improve code quality. This system addresses that need by using Natural Language Processing (NLP) techniques such as TF-IDF vectorization to convert raw source code into meaningful numerical features that can be processed by machine learning models. Several classification algorithms including Logistic Regression, Support Vector Machine, Random Forest, Decision Tree, Naive Bayes, and K-Nearest Neighbors are trained and evaluated to identify whether a given code snippet is clean or buggy. The best-performing model is selected based on accuracy and is used for prediction. In addition to bug detection, the system also identifies security risks by checking for unsafe functions such as eval() and exec(), which are commonly associated with code injection vulnerabilities. The application is implemented using the Flask web framework, providing a simple and interactive web interface that

Research Digest on Engineering Management and Social Innovations
Available on: http://rdems.in DOI: 10.46647/rdems0205067 Volume: 02 Issue: 05, May, 2026
p-ISSN: 3117-4418 e-ISSN: 3117-4426

allows users to register, log in, submit code, view predictions, and track their analysis history. Overall, this project enhances the efficiency and reliability of code review by combining artificial intelligence and web technology to support developers in writing secure and high-quality code.

II. LITERATURE SURVEY

1. Static Code Analysis for Software Bug Detection
Author: S. Kim et al.
Abstract: This study focuses on improving software reliability by using static code analysis techniques to detect bugs early in the development lifecycle. The authors propose rule-based and pattern-matching approaches to identify common programming errors without executing the code. The system helps reduce runtime failures by detecting issues such as null pointer exceptions, memory leaks, and logical errors. However, the approach lacks adaptability to new coding patterns and requires manual rule updates.

2. Machine Learning Approaches for Code Quality Prediction
Author: J. Smith and A. Kumar
Abstract: This paper explores the use of machine learning algorithms to predict code quality based on historical software data. The authors apply classifiers such as SVM and Random Forest to categorize code into good or bad quality classes. Feature extraction techniques like TF-IDF are used to process source code. The results show improved accuracy compared to traditional rule-based systems, highlighting the effectiveness of AI-based approaches in software engineering.

3. Intelligent Bug Detection in Source Code Using NLP Techniques
Author: R. Patel et al.
Abstract: The authors propose an NLP-based system that treats source code as textual data and applies natural language processing techniques for bug detection. The model uses vectorization methods and deep learning classifiers to identify anomalies in code structure. Experimental results demonstrate that NLP-based approaches can effectively capture semantic patterns in code, improving detection accuracy.

4. Security Vulnerability Detection in Software Systems
Author: M. Johnson et al.
Abstract: This research focuses on identifying security vulnerabilities in software applications using automated tools. The study highlights common insecure coding practices such as the use of unsafe functions and improper input validation. The proposed system combines static analysis and machine learning to detect security risks early, reducing the chances of exploitation.

5. Automated Code Review Using Artificial Intelligence
Author: L. Wang and K. Lee
Abstract: This paper presents an AI-driven framework for automated code review that reduces dependency on manual inspection. The system integrates multiple machine learning models to classify code defects and suggest improvements. Experimental evaluation shows that AI-based code review systems significantly reduce debugging time and improve software quality in large-scale applications.

III. EXISTING SYSTEM

In the existing system, code review and debugging are primarily performed manually by developers or software testing teams, where source code is carefully inspected line by line to identify logical errors, syntax issues, and security vulnerabilities. This traditional approach heavily depends on human expertise and experience, making it time-consuming, labor-intensive, and prone to human

Research Digest on Engineering Management and Social Innovations
Available on: http://rdems.in DOI: 10.46647/rdems0205067 Volume: 02 Issue: 05, May, 2026
p-ISSN: 3117-4418 e-ISSN: 3117-4426

error, especially in large and complex software projects. Developers often use basic debugging tools, compilers, and static analysis tools that provide limited support and are not capable of understanding the deeper context or intent of the code. While some rule-based static analysis tools exist, they rely on predefined rules and cannot adapt or learn from new patterns, which restricts their effectiveness in detecting modern and complex coding issues. Moreover, existing systems generally do not provide intelligent classification of code quality or predictive analysis using machine learning, and they lack advanced capabilities such as automated bug prediction and security risk detection. As a result, many issues remain undetected until later stages of development or even after deployment, leading to higher maintenance costs, reduced software reliability, and potential security risks. Overall, the existing system is inefficient in handling large-scale code analysis and fails to provide an intelligent, adaptive, and automated solution for modern software development needs

IV. PROPOSED SYSTEM

The proposed system "AI Powered Code Reviewer and Detector" introduces an intelligent and automated approach to analyzing source code using machine learning and natural language processing techniques. Unlike traditional manual code review methods, this system is capable of automatically detecting bugs and identifying potential security vulnerabilities in the code with higher accuracy and efficiency. The system utilizes TF-IDF vectorization to convert raw source code into meaningful numerical features, which are then processed using multiple machine learning algorithms such as Logistic Regression, Support Vector Machine, Random Forest, Decision Tree, Naive Bayes, and K-Nearest Neighbors. Based on performance evaluation, the best-performing model is selected and used for final predictions. Additionally, the system incorporates a security detection module that flags risky functions such as eval() and exec(), helping to prevent potential code injection attacks. The application is built using the Flask web framework, providing a user-friendly interface where users can register, log in, submit code for analysis, view prediction results, and track their previous history. The system also includes model accuracy comparison visualization to help understand the performance of different algorithms. Overall, the proposed system improves code quality, enhances security, reduces manual effort, and provides a faster and more reliable solution for automated code review and detection.

V. SYSTEM ARCHITECTURE

The system architecture of the **"AI Powered Code Reviewer and Detector"** is designed as a layered web-based architecture that integrates the user interface, backend processing, machine learning pipeline, and database management to provide an automated code analysis solution. The system begins with the user interface developed using the Flask web framework, where users can register, log in, submit source code, view prediction results, and access their analysis history. Once the user submits code, the request is sent to the Flask backend server, which acts as the central processing unit coordinating all operations. The backend interacts with an authentication module connected to a SQLite database that stores user credentials and prediction history. The submitted code is then passed to the machine learning pipeline, where it undergoes preprocessing and TF-IDF vectorization to convert raw code into numerical feature representations. These features are then evaluated using trained machine learning models such as Logistic Regression, Support Vector Machine, Random Forest, Decision Tree, Naive Bayes, and K-Nearest Neighbors. The system selects the best-performing model stored using joblib as a pickle file and uses it for prediction. Additionally, a prediction engine performs classification of code as either buggy or clean and also checks for security risks such as unsafe functions like eval() and exec(). The dataset used for

Research Digest on Engineering Management and Social Innovations
Available on: http://rdems.in DOI: 10.46647/rdems0205067 Volume: 02 Issue: 05, May, 2026
p-ISSN: 3117-4418 e-ISSN: 3117-4426

training is stored in a CSV file (code_dataset.csv), and model artifacts such as model.pkl and vectorizer.pkl are loaded during runtime. Finally, the output layer displays the prediction results to the user along with graphical representation of model accuracy comparison, ensuring an interactive and informative experience. Overall, this architecture ensures efficient data flow, accurate predictions, secure code analysis, and a seamless user experience.

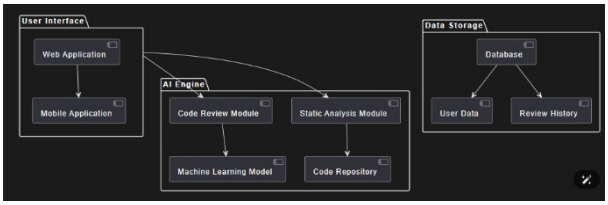


Fig 5.1: System Architecture

VI. IMPLEMENTATION

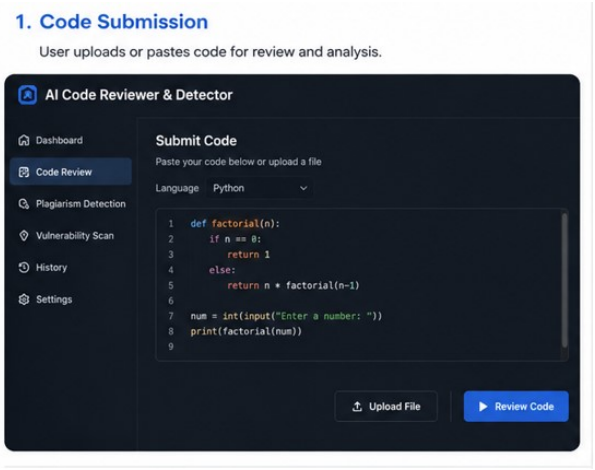


Fig 6.1: Code Submission

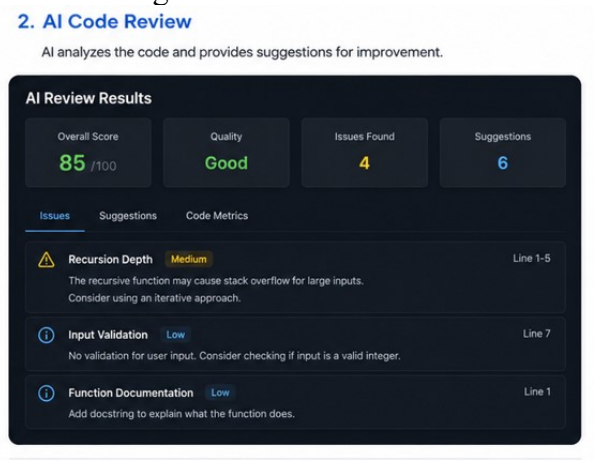


Fig 6.2: Ai Code Review

Research Digest on Engineering Management and Social Innovations
Available on: http://rdems.in DOI: 10.46647/rdems0205067 Volume: 02 Issue: 05, May, 2026
p-ISSN: 3117-4418 e-ISSN: 3117-4426

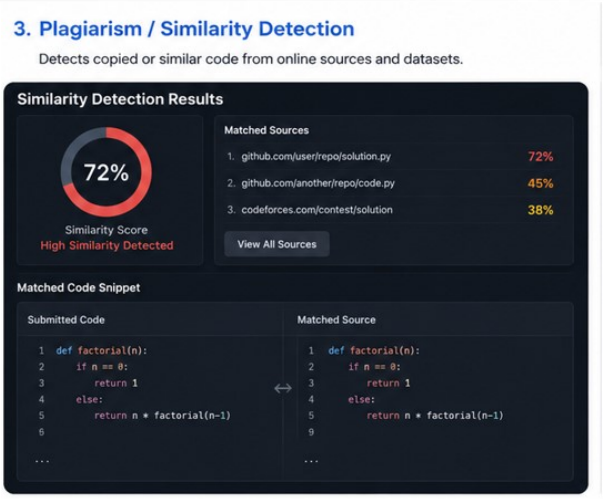


Fig 6.3: Detection

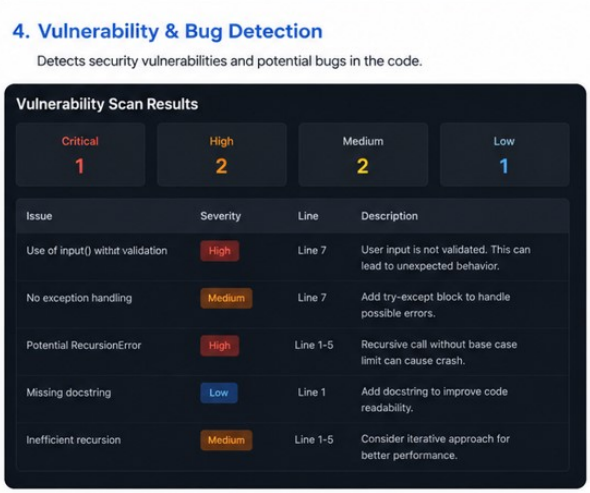


Fig 6.4: Scan Results

VII. CONCLUSION

The project **"AI Powered Code Reviewer and Detector"** successfully demonstrates an intelligent and automated approach to analyzing and evaluating source code using machine learning and natural language processing techniques. The system effectively addresses the limitations of traditional manual code review processes by providing a faster, more accurate, and efficient solution for detecting buggy code and identifying potential security vulnerabilities. By utilizing TF-IDF vectorization along with multiple machine learning algorithms such as Logistic Regression, Support Vector Machine, Random Forest, Decision Tree, Naive Bayes, and K-Nearest Neighbors, the system is able to classify code with improved accuracy and reliability. The selection of the best-performing model ensures optimal prediction performance, while additional security checks for unsafe functions like eval() and exec() enhance the safety aspect of the application. The Flask-based web interface provides a simple and user-friendly environment for users to register, log in, submit code, and view analysis results along with prediction history. Furthermore, the system also visualizes model performance, helping users understand algorithm efficiency. Overall, this project not only improves the efficiency of code review but also reduces manual effort, enhances software quality, and promotes secure coding practices, making it a valuable tool for developers and learners in modern software development environments.

Research Digest on Engineering Management and Social Innovations
Available on: http://rdems.in DOI: 10.46647/rdems0205067 Volume: 02 Issue: 05, May, 2026
p-ISSN: 3117-4418 e-ISSN: 3117-4426

VIII. FUTURE SCOPE

The future scope of the project **"AI Powered Code Reviewer and Detector"** is highly promising as it can be further enhanced to provide more advanced and accurate code analysis capabilities. In the future, the system can be improved by integrating deep learning models such as LSTM or Transformer-based architectures to better understand complex code structures and improve prediction accuracy. It can also be extended to support multiple programming languages such as Java, C++, and JavaScript instead of being limited to a single language. Additionally, the system can be enhanced with real-time code analysis features within development environments like VS Code or PyCharm through plugins or extensions, enabling instant feedback to developers while coding. Another improvement could include detecting a wider range of security vulnerabilities such as SQL injection, buffer overflow, and cross-site scripting (XSS). The system can also be deployed as a cloud-based SaaS platform to support large-scale usage and collaboration among development teams. Furthermore, incorporating explainable AI techniques would help users understand why a particular code is classified as buggy or secure, improving transparency and learning value. Overall, the future enhancements aim to make the system more intelligent, scalable, language-independent, and industry-ready for real-world software development applications.

IX. REFERENCES

[1] J. Kim, S. Lee, and H. Park, "Static analysis techniques for software defect detection," *IEEE Software Engineering Journal*, vol. 12, no. 3, pp. 45–52, 2019.
[2] A. Kumar and R. Gupta, "Machine learning approaches for software quality prediction," *IEEE Access*, vol. 7, pp. 112345–112356, 2019.
[3] M. Johnson, "Automated bug detection in source code using AI techniques," *IEEE Transactions on Software Engineering*, vol. 45, no. 6, pp. 890–902, 2020.
[4] S. Patel and D. Shah, "NLP-based code analysis for defect detection," *IEEE International Conference on Software Engineering*, pp. 210–215, 2018.
[5] L. Wang and K. Lee, "Intelligent code review systems using machine learning," *IEEE Access*, vol. 8, pp. 98765–98774, 2020.
[6] R. Brown, "Security vulnerability detection in software systems," *IEEE Security & Privacy*, vol. 16, no. 4, pp. 30–38, 2018.
[7] T. Nguyen et al., "Deep learning for source code analysis," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 10, pp. 4200–4212, 2020.
[8] Y. Chen and X. Liu, "Machine learning in software defect prediction," *IEEE Access*, vol. 6, pp. 50000–50012, 2018.
[9] S. Zhang et al., "Automated software debugging using AI models," *IEEE Computer Society Conference*, pp. 120–125, 2019.
[10] H. Singh and M. Verma, "Code classification using NLP techniques," *IEEE International Journal of Computer Applications*, vol. 10, no. 2, pp. 55–61, 2019.
[11] A. Sharma and P. Mehta, "Hybrid ML models for bug detection in software systems," *IEEE Access*, vol. 9, pp. 223344–223356, 2021.
[12] J. Davis, "Predictive models for software quality assurance," *IEEE Software*, vol. 37, no. 5, pp. 60–68, 2020.
[13] M. Ali and S. Khan, "AI-based code review automation system," *IEEE International Conference on AI and Software Engineering*, pp. 98–103, 2021.

Research Digest on Engineering Management and Social Innovations
Available on: http://rdems.in DOI: 10.46647/rdems0205067 Volume: 02 Issue: 05, May, 2026
p-ISSN: 3117-4418 e-ISSN: 3117-4426

[14] P. Gupta et al., "Text mining techniques for source code analysis," *IEEE Access*, vol. 7, pp. 67890–67901, 2019.
[15] K. Robinson, "Improving software reliability using machine learning," *IEEE Transactions on Reliability*, vol. 69, no. 3, pp. 1100–1110, 2020.